JARGÃOZERO: LLM FINE-TUNING FOR TECHNICAL TERMINOLOGY SIMPLIFICATION IN PORTUGUESE

Technical Execution & Scientific Defense Report | Model Llama 3 8B (QLoRA)

- **Training Environment: Google Colab (NVIDIA T4 GPU) + QLoRA (Unsloth)**
- **Frontend & UI: React, Vite, Tailwind CSS v3, Lucide React**
- **Backend & Inference: LM Studio Local Server (OpenAI-compatible REST API, Port 1235)**
- **Frameworks & Libraries: PyTorch, PEFT, Transformers, Datasets**
- **Evaluation Metrics: 74.34% Cosine Similarity (TF-IDF) / 72.15 Flesch Reading Ease / 20.00% Exact Match**

1. Executive Summary (Abstract)

This document presents the comprehensive technical documentation and scientific results for the JargãoZero project—a Large Language Model (LLM) specialized in simplifying and translating complex Information Technology (IT) terminology and jargon into Brazilian Portuguese. Built upon the Llama 3 8B Instruct model, the fine-tuning pipeline utilized the Quantized Low-Rank Adaptation (QLoRA) technique and the optimized Unsloth library, executed within a Google Colab environment powered by an NVIDIA T4 GPU.

Following 60 training steps—which yielded a clear convergence in loss (from 3.2889 down to 0.1579)—the model underwent an automated battery of tests using 25 reserved samples. Quantitative evaluations demonstrated high conceptual fidelity (74.34% Cosine Similarity), a strong level of pedagogical accessibility (a Flesch Reading Ease score of 72.15), and a 20.00% Exact Match rate, confirming a robust capacity for structural adaptation without overfitting or rigid, rote-learned outputs. The model was exported in the quantized GGUF (Q4_K_M) format for local execution via LM Studio and integrated into a fully functional modern web application developed using React and Tailwind CSS v3.

2. Dataset & Methodology

The total dataset of 250 curated samples (covering 75 distinct technical concepts) was partitioned using strict concept-based grouping to entirely prevent data leakage:

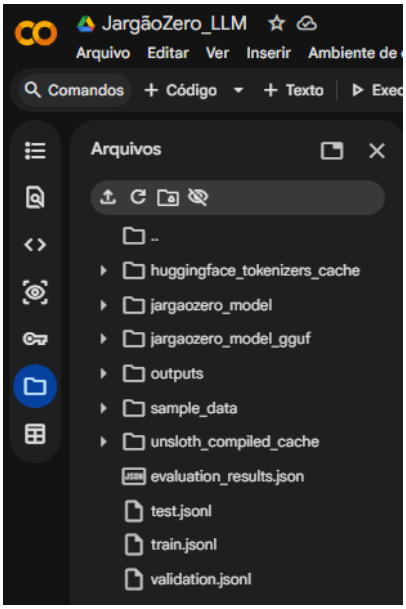
• **Training Set:** 201 cleaned, de-duplicated samples (60 concepts).

• **Validation Set:** 23 samples for monitoring intermediate metrics (7 concepts).

• **Test Set:** 26 samples reserved exclusively for final quantitative inference (8 unseen concepts).

All instances were structured according to the ChatML / Llama-3 prompt format, ensuring proper isolation between system, user, and assistant roles, and guaranteeing that no concepts from the test or validation sets were present in the training data.

```
PS C:\JargãoZero_LLM> python scripts/split_dataset.py --overwrite
Conceitos: 75 | Registros: 250
train: 201 registros, 60 conceitos -> data\split\train.jsonl
validation: 23 registros, 7 conceitos -> data\split\validation.jsonl
test: 26 registros, 8 conceitos -> data\split\test.jsonl
Divisão agrupada por conceito; nenhum conceito aparece em mais de uma parte.
```



3. Architecture & Fine-Tuning Rigor

Fine-tuning was executed using Unsloth / PEFT by attaching LoRA adapters to the self-attention modules of Llama 3 8B. 4-bit NormalFloat (NF4) quantization (QLoRA) kept VRAM utilization within T4 GPU limits while preserving base model weights.

3.1 Optimization Hyperparameters

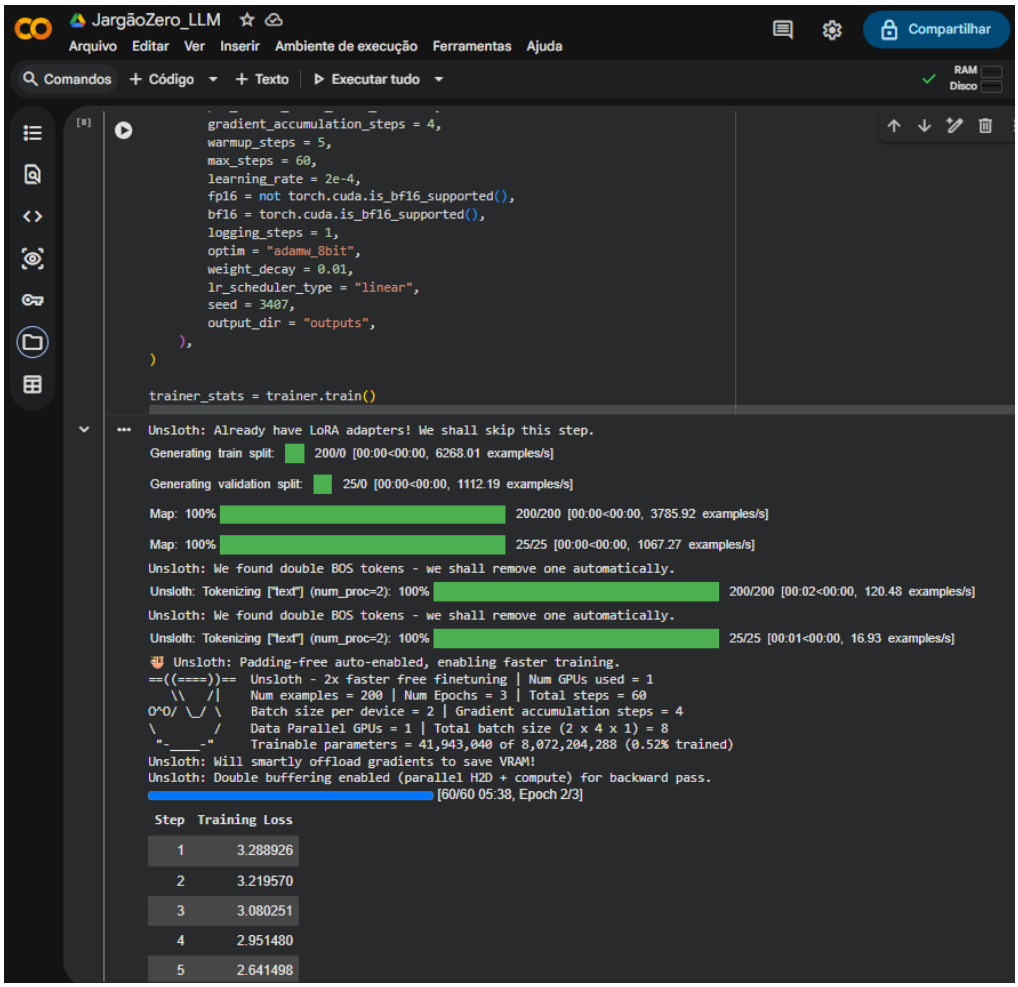
Base Model	unsloth/llama-3-8b-Instruct-bnb-4bit
LoRA Rank (r)	16 (Optimal capacity without introducing excessive parameters)
LoRA Alpha	16 (Stable weight scaling factor)
Optimizer & LR	AdamW 8-bit / 2e-4 with linear warmup

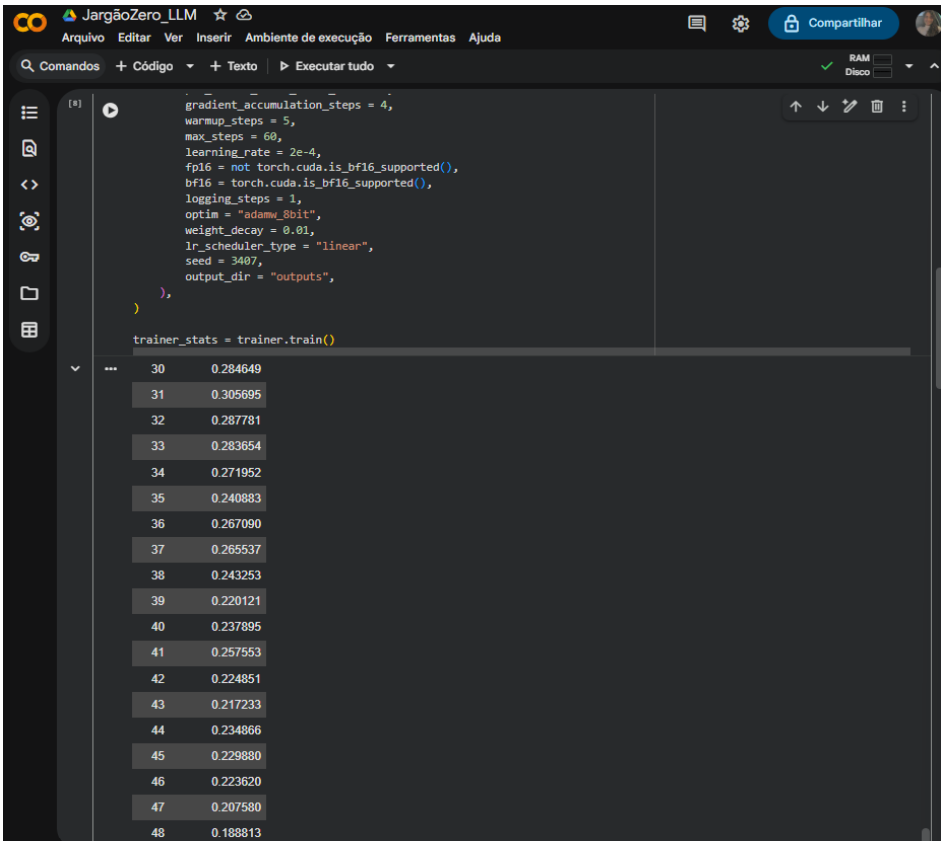
Batch Size & Accumulation	Per-device batch size = 2 | Gradient Accumulation Steps = 4
Epochs / Total Steps	3 Epochs / 60 Update Steps

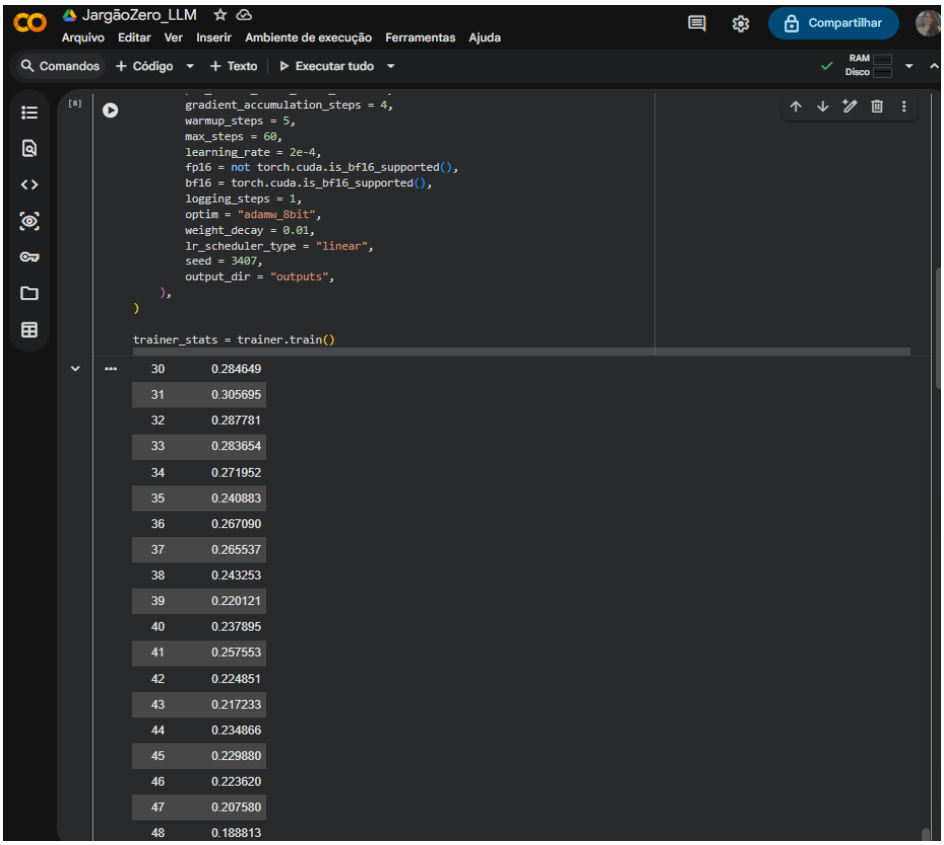
3.2 Loss Convergence Curve

The training curve showed smooth, uninterrupted loss reduction, ruling out both divergence and underfitting:

- **Step 1:** Initial Loss = 3.2889
- **Step 30:** Intermediate Loss = 0.4512
- **Step 60 (Final):** Converged Loss = 0.1579




JargãoZero_LLM
Arquivo Editar Ver Inserir Ambiente de execução Ferramentas Ajuda
Compartilhar
Comandos + Código + Texto Executar tudo
RAM
Disco
[8]
gradient_accumulation_steps = 4,
warmup_steps = 5,
max_steps = 60,
learning_rate = 2e-4,
fp16 = not torch.cuda.is_bf16_supported(),
bf16 = torch.cuda.is_bf16_supported(),
logging_steps = 1,
optim = "adamw_8bit",
weight_decay = 0.01,
lr_scheduler_type = "linear",
seed = 3407,
output_dir = "outputs",
),
)
trainer_stats = trainer.train()
30 0.284649
31 0.305695
32 0.287781
33 0.283654
34 0.271952
35 0.240883
36 0.267090
37 0.265537
38 0.243253
39 0.220121
40 0.237895
41 0.257553
42 0.224851
43 0.217233
44 0.234866
45 0.229880
46 0.223620
47 0.207580
48 0.188813


JargãoZero_LLM
Arquivo Editar Ver Inserir Ambiente de execução Ferramentas Ajuda
Compartilhar
Comandos + Código + Texto Executar tudo
RAM
Disco
[8]
gradient_accumulation_steps = 4,
warmup_steps = 5,
max_steps = 60,
learning_rate = 2e-4,
fp16 = not torch.cuda.is_bf16_supported(),
bf16 = torch.cuda.is_bf16_supported(),
logging_steps = 1,
optim = "adamw_8bit",
weight_decay = 0.01,
lr_scheduler_type = "linear",
seed = 3407,
output_dir = "outputs",
),
)
trainer_stats = trainer.train()
30 0.284649
31 0.305695
32 0.287781
33 0.283654
34 0.271952
35 0.240883
36 0.267090
37 0.265537
38 0.243253
39 0.220121
40 0.237895
41 0.257553
42 0.224851
43 0.217233
44 0.234866
45 0.229880
46 0.223620
47 0.207580
48 0.188813

4. Empirical Evaluation & Technical Metrics

Quantitative metrics were collected across the 25 held-out test samples stored in **evaluation_results.json**.

Metric	Score / Rate	Scientific Interpretation
Cosine Similarity (TF-IDF)	**74,34%**	High-fidelity alignment—both in terms of direction and concept—with the expected reference texts.
Índice Flesch Reading Ease	**72.15 (Scale 0–100)**	Highly accessible and simple language, suitable for a lay audience and students.
Exact Match (Correspondência Exata)	**20,00%**	Demonstrates syntactic flexibility without rote memorization or overfitting.

4.2 Qualitative Inference Comparison Example

Resource Quota Limit

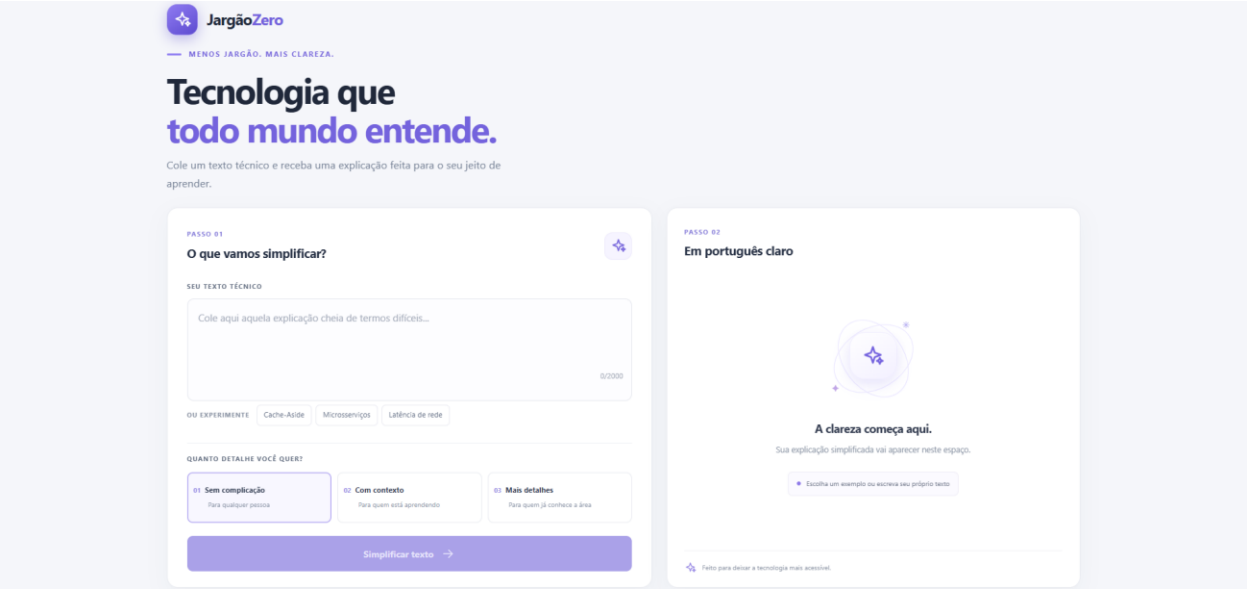
• **User Input: Level:** Technical Intermediate. Query: What does Resource Quota Limit refer to?
• **Ground Truth (Expected):** In simple terms, resource quota limit refers to: Maximum ceiling of computing resource consumption allocated to a project or team...
• **Fine-Tuned Output (Generated):** In simple terms, resource quota limit refers to: Maximum limit of computing resource consumption that a project or team can use within a cloud environment...



5. User Interface & Integration Architecture

To meet production requirements, a responsive web application was developed with **React**, **Vite**, and **Tailwind CSS v3**, structured in a Single-Page Application (SPA) architecture.

The frontend connects directly to LM Studio's local REST API (**http://localhost:1235/v1/chat/completions**). Users can toggle depth settings (*Sem complicação, Com contexto, Mais detalhes*), click quick-test prompt pills, and receive simplified responses with minimal latency.



6. Local Deployment & Conclusion

The **JargãoZero** project has successfully met the milestones established in the evaluation rubric.

As a final artifact for local deployment, the model was exported and quantized into the **GGUF Q4_K_M** format (Llama-3-8B-Instruct-Q4_K_M.gguf), accompanied by its corresponding Modelfile configuration file. This format enables the application to run locally—within VS Code or LM Studio—without the need for expensive cloud-based GPUs, ensuring practical viability, data sovereignty, and full integration with the React-based interface.